

STM32F446 I²C1 Polling Driver — V1 Architecture and Design Note

1. Purpose

This driver provides a deterministic, register-level I²C controller implementation for the STM32F446RE.

The V1 driver supports:

- 7-bit target addressing
- Controller/master operation
- Polling-based transactions
- Single-byte and multi-byte writes
- One-byte, two-byte and multi-byte reads
- Combined write-read transactions using a repeated START
- Bounded software timeouts
- Driver state management
- NACK cleanup
- Peripheral-level recovery
- Basic hardware-bus admission checking

The implementation uses CMSIS peripheral definitions and direct register access. It does not use STM32 HAL or LL drivers.

The public interface and current implementation are defined in `i2c_driver.h` and `i2c_driver.c`.

2. V1 Operating Contract

The polling V1 driver is intentionally limited to a simple execution model.

Assumptions

- Only one software context uses the driver.
- The driver is not reentrant.
- Public I²C functions are not called from interrupt service routines.
- The system has only one active I²C controller.
- External pull-up resistors are fitted to SDA and SCL.
- The APB1 peripheral clock is 45 MHz.
- I²C1 uses PB6 for SCL and PB7 for SDA.
- Both pins use alternate function AF4.
- Only the accepted standard-mode timing configuration is supported.
- Fast mode is reserved for a later version.

- Timeouts are software loop counters rather than real-time deadlines.

These limitations simplify V1 while preserving an architecture that can later be converted to interrupt-driven or DMA-based operation.

3. Architectural Layers

The driver is divided into four conceptual layers.

3.1 Public transaction managers

The public functions manage complete application-visible operations:

```
i2c_init()  
i2c_write()  
i2c_read()  
i2c_write_read()  
i2c_recover()  
i2c_get_state()
```

Their responsibilities are:

- Validate public parameters.
- Check the software driver state.
- Check initial physical-bus availability.
- Acquire ownership of the driver.
- Select the correct protocol sequence.
- Apply final transaction-state policy.
- Decide whether the driver returns to **READY** or enters **ERROR**.

The public transaction managers do not directly implement every register-level event.

3.2 Transaction and sequence helpers

These functions implement complete protocol phases:

```
i2c_transmit_phase()  
i2c_receive_1_byte()  
i2c_receive_2_bytes()  
i2c_receive_n_bytes()
```

```
i2c_complete_transaction()
i2c_abort_sequence()
```

Their responsibilities include:

- Correct START, address and data ordering.
- Correct ADDR flag clearing.
- Correct ACK and POS configuration.
- Correct placement of STOP.
- Correct handling of the final bytes of a receive transaction.
- Ending or abandoning an unsuccessful hardware sequence.

The receive operation is divided into separate one-byte, two-byte and N-byte functions because the STM32F4 I²C peripheral requires different closing sequences for these cases.

3.3 Register-level primitives

The primitive workers perform small hardware operations:

```
i2c_generate_start()
i2c_generate_stop()
i2c_send_address()
i2c_send_byte()
i2c_receive_byte()
i2c_read_dr()
i2c_clear_addr_flag()
i2c_enable_ack()
i2c_disable_ack()
i2c_enable_pos()
i2c_disable_pos()
i2c_wait_sr1_flag()
i2c_clear_error_flags()
i2c_terminate_bus_request()
```

These functions own the STM32-specific register mechanisms.

For example:

```
i2c_generate_stop()
```

only sets `CR1.STOP`. It does not wait for STOP completion or change the software state.

This separation is intentional:

Primitive:

Request a hardware action.

Sequence helper:

Place that action at the correct protocol point.

Public manager:

Decide the final software state and return status.

3.4 Driver context

The V1 driver has one internal context:

```
static i2c_state_t s_state;  
static i2c_config_t s_config;  
static uint8_t s_config_valid;
```

`s_state` represents the current software lifecycle.

`s_config` stores the most recently validated and successfully applied configuration.

`s_config_valid` prevents recovery before a successful initialization.

The stored configuration allows `i2c_recover()` to reconfigure I²C1 without requiring the application to pass the configuration again.

4. Driver State Machine

The V1 states are:

```
I2C_STATE_RESET  
I2C_STATE_READY  
I2C_STATE_BUSY_TX  
I2C_STATE_BUSY_RX  
I2C_STATE_BUSY_TX_RX  
I2C_STATE_ERROR
```

State meanings

`I2C_STATE_RESET`

The driver has not completed initialization.

Public transactions return:

```
I2C_ERR_NOT_INIT
```

`I2C_STATE_READY`

The driver is available for a new operation.

Before changing from `READY` to a busy state, the manager also checks the hardware `SR2.BUSY` flag.

`I2C_STATE_BUSY_TX`

A pure write transaction is active.

`I2C_STATE_BUSY_RX`

A pure read transaction is active.

`I2C_STATE_BUSY_TX_RX`

A combined write-read transaction is active.

This separate state prevents another public operation or recovery request from interrupting the repeated-START transaction.

`I2C_STATE_ERROR`

A failure occurred and the peripheral state can no longer be trusted.

New read/write operations return:

```
I2C_ERR_LOCKED
```

The application must call:

```
i2c_recover()
```

before attempting another transaction.

5. Initialization Flow

The public initialization sequence is:

```
Validate configuration
    ↓
Set state to RESET
Invalidate saved configuration
    ↓
Enable GPIOB and I2C1 clocks
    ↓
Configure PB6 and PB7
    ↓
Reset I2C1 through RCC
    ↓
Configure timing and control registers
    ↓
Enable I2C1
    ↓
Establish ACK/POS baseline
    ↓
Save validated configuration
    ↓
Set state to READY
```

GPIO configuration

PB6 and PB7 are configured as:

- Alternate-function mode
- AF4
- Open-drain outputs
- Very-high-speed GPIO setting
- No internal pull-up or pull-down

External pull-up resistors are assumed.

Peripheral reset

The private function:

```
i2c_reset_and_configure_peripheral()
```

resets only I2C1 using `RCC_APB1RSTR`.

It then:

- Ensures the peripheral is disabled.
- Sets the required OAR1 bit 14.
- Applies the timing configuration.
- Enables the peripheral.
- Enables ACK.
- Clears POS.

This private function is shared by initialization and recovery.

The public functions remain independent:

```
i2c_init()  
    Configures clocks, GPIO and I2C1.
```

```
i2c_recover()  
    Reconfigures only I2C1.
```

`i2c_recover()` does not call `i2c_init()`, and `i2c_init()` does not call `i2c_recover()`.

6. Timing Decision

The accepted V1 configuration is:

```
#define I2C_CLOCK_STANDARD_SAFE_HZ 88000UL
```

The application provides it through:

```
const i2c_config_t i2c_config =  
{  
    .clock_speed = I2C_CLOCK_STANDARD_SAFE_HZ  
};  
  
status = i2c_init(&i2c_config);
```

The application chooses the requested bus configuration. The driver validates and implements it.

Ceiling division

The CCR value is calculated using ceiling division:

```
ccr = (APB1_CLOCK_HZ +  
      (2UL * clock_speed) - 1UL) /  
      (2UL * clock_speed);
```

This prevents integer truncation from producing a bus frequency above the requested value.

With:

```
APB1 clock = 45 MHz  
Requested frequency = 88 kHz  
CCR = 256
```

the actual nominal SCL frequency is approximately:

```
87.89 kHz
```

The conservative frequency was selected for the V1 repeated-START implementation.

Fast-mode support remains explicitly rejected by `i2c_configure_timing()` and is reserved for a future version.

7. Transaction Admission

Software readiness does not prove physical-bus readiness.

Therefore, each public transaction checks both:

```
Software state = READY  
Hardware SR2.BUSY = 0
```

The admission pattern is:


```

case I2C_STATE_READY:

    if ((I2C1->SR2 & I2C_SR2_BUSY) != 0U)
    {
        s_state = I2C_STATE_ERROR;
        return I2C_ERR_BUS;
    }

    s_state = required_busy_state;
    break;

```

This prevents the driver from requesting an initial START when the peripheral already detects a busy or stuck bus.

In the V1 single-controller system, the combination:

```

Software state = READY
Hardware BUSY = 1

```

is treated as an inconsistency or physical-bus fault.

This check is performed only before the initial START.

It is not placed inside `i2c_generate_start()`, because that function is also used to generate the repeated START during `i2c_write_read()`.

During a valid repeated START:

```

SR2.BUSY = 1
SR2.MSL  = 1

```

because the controller intentionally still owns the bus.

8. Pure Write Transaction

The public write manager performs:

```

Validate address, pointer and length
    ↓
Check driver state and physical BUSY

```

```
↓
Set BUSY_TX
↓
Call i2c_transmit_phase()
↓
Generate STOP
↓
Complete transaction
↓
Set READY or ERROR
```

Transmit phase

`i2c_transmit_phase()` performs:

```
START
↓
Wait for SB
↓
Send address + write
↓
Wait for ADDR
↓
Clear ADDR
↓
Wait for TXE and write each byte
↓
Wait for final BTF
```

It deliberately does not generate STOP.

This allows the same transmit phase to be reused by:

```
i2c_write()
i2c_write_read()
```

The public caller chooses what happens after the final BTF:

```
Pure write:
    Generate STOP.
```

Combined write-read:
Generate repeated START through the receive helper.

Why final BTF is required

TXE means that the data register is empty. It does not guarantee that the final byte has finished shifting on SDA.

BTF confirms that:

- The data register is empty.
- The shift register has completed the byte transfer.

Therefore, STOP or repeated START is requested only after final BTF.

9. Pure Read Transaction

The public read manager selects one of three receive sequences based on length:

```
length == 1
    ↪ i2c_receive_1_byte()

length == 2
    ↪ i2c_receive_2_bytes()

length > 2
    ↪ i2c_receive_n_bytes()
```

This dispatch is necessary because the STM32F4 peripheral requires different ACK, POS and STOP timing for each case.

9.1 One-byte receive

The one-byte sequence is:

```
Set ACK = 1 and POS = 0 baseline
    ↓
START
    ↓
Address + read
    ↓
```

```
Wait for ADDR
↓
Disable ACK before clearing ADDR
↓
Enter critical section
↓
Clear ADDR
↓
Request STOP immediately
↓
Leave critical section
↓
Wait for RXNE
↓
Read DR
```

The critical section prevents an interrupt from creating excessive delay between:

```
Clear ADDR
Request STOP
```

The previous interrupt-enable condition is preserved through `PRIMASK`.

9.2 Two-byte receive

The two-byte sequence is:

```
Set ACK = 1 and POS = 0 baseline
↓
START
↓
Address + read
↓
Wait for ADDR
↓
Set POS = 1
↓
Clear ACK
↓
Clear ADDR
↓
Wait for BTF
↓
```

```
Request STOP
↓
Read DR twice
```

POS is used so that the NACK is applied correctly to the second received byte.

After transaction completion, POS is restored to zero.

9.3 More-than-two-byte receive

The N-byte sequence is divided into a bulk phase and a final-three-byte teardown.

Bulk phase

While more than three bytes remain:

```
Wait for RXNE
Read DR
Advance buffer pointer
Decrease remaining length
```

Final three bytes

```
Wait for BTF
↓
Disable ACK
↓
Read byte N-2
↓
Wait for BTF
↓
Request STOP
↓
Read byte N-1
↓
Wait for RXNE
↓
Read byte N
```

The final-three-byte sequence ensures that the final byte is NACKed and STOP is generated at the correct point.

10. Combined Write-Read Transaction

The public function:

```
i2c_write_read()
```

supports common register-based target transactions such as:

```
Write register address  
Repeated START  
Read register data
```

Its bus sequence is:

```
START  
  ↓  
Address + write  
  ↓  
Transmit command/register bytes  
  ↓  
Wait for final BTF  
  ↓  
Repeated START  
  ↓  
Address + read  
  ↓  
Receive 1, 2 or N bytes  
  ↓  
STOP
```

No STOP is generated between the write and read phases.

Implementation reuse

The combined transaction calls:

```
i2c_transmit_phase()
```

which leaves the controller owning the bus after final BTF.

It then dispatches to the existing receive helper.

Each receive helper begins by setting `CR1 .START` .

Because the controller still owns a busy bus, this START request becomes a repeated START rather than an initial START.

This avoids duplicating receive functions such as:

```
i2c_receive_1_byte_after_restart()  
i2c_receive_2_bytes_after_restart()  
i2c_receive_n_bytes_after_restart()
```

The same receive sequences are valid after both an initial START and a repeated START.

The combined operation owns the driver through:

```
I2C_STATE_BUSY_TX_RX
```

for the complete write and read sequence.

11. Transaction Completion

The receive helpers generate STOP at the length-specific protocol point.

The pure write manager generates STOP after final BTF.

After STOP has been requested, the public manager calls:

```
i2c_complete_transaction()
```

The completion worker:

- Waits until the hardware clears `CR1 .STOP` .
- Verifies that controller ownership has ended.
- Restores ACK.
- Clears POS.

The final reviewed design should verify both:

```
I2C_SR2_MSL
I2C_SR2_BUSY
```

before declaring completion:

```
if ((sr2 & (I2C_SR2_MSL | I2C_SR2_BUSY)) != 0U)
{
    return I2C_ERR_BUS;
}
```

This preserves the invariant:

```
I2C_STATE_READY means:
    no software transaction is active,
    the peripheral is no longer controller,
    and the physical bus is idle.
```

12. Error Statuses

The V1 status values distinguish the main failure classes:

```
I2C_OK
I2C_ERR_BUSY
I2C_ERR_LOCKED
I2C_ERR_NOT_INIT
I2C_ERR_TIMEOUT
I2C_ERR_NACK
I2C_ERR_PARAM
I2C_ERR_BUS
I2C_ERR_ARBITRATION_LOST
I2C_ERR_OVERRUN
```

Meaning of important statuses

`I2C_ERR_BUSY`

Another driver transaction is active.

This represents a software ownership conflict.

`I2C_ERR_LOCKED`

The driver is in `I2C_STATE_ERROR`.

The application must call `i2c_recover()`.

`I2C_ERR_NACK`

The target did not acknowledge an address or data byte.

NACK is treated as a transaction-level result rather than immediate evidence of a damaged peripheral.

`I2C_ERR_BUS`

The controller or physical bus is not in a usable state.

Examples include:

- Hardware BUSY detected while admitting a new transaction.
- BUSY remaining after recovery.
- STOP completing without the peripheral releasing bus ownership.
- Another unclassified controller-bus inconsistency.

`I2C_ERR_ARBITRATION_LOST`

The ARLO flag was detected.

The error is kept separate from a general bus error for diagnostics and future retry policy.

`I2C_ERR_OVERRUN`

The OVR flag was detected.

The error is kept separate because it indicates that receive or transmit data handling did not keep pace with the peripheral.

13. Flag Waiting and Error Priority

The common polling function:

```
i2c_wait_sr1_flag()
```

takes an SR1 snapshot on each iteration.

It checks error conditions before accepting the requested success flag.

The processing order is:

```
Read SR1
↓
Handle spurious BERR policy
↓
Check ARLO
↓
Check AF/NACK
↓
Check OVR
↓
Check requested success flag
↓
Decrease timeout
```

Error conditions take priority over success because multiple SR1 bits may be set in the same snapshot.

The BERR condition is cleared and ignored under the accepted STM32F446 controller-mode erratum policy.

The timeout is currently:

```
#define I2C_TIMEOUT_COUNT 100000UL
```

This is an iteration count, not a fixed duration.

Its real time depends on:

- CPU frequency
- Compiler optimization
- Register access latency
- Code placement
- Debug or release build configuration

A later driver version should replace it with a timer or system-tick deadline without changing the transaction architecture.

14. Abort and NACK Policy

A failed sequence calls:

```
i2c_abort_sequence(error_status)
```

which calls:

```
i2c_terminate_bus_request()
```

When the peripheral still owns the bus

If `SR2.MSL` is set:

```
Request STOP
```

When ownership was never obtained or was lost

If `SR2.MSL` is clear:

```
Clear any outstanding START request
```

The sequence helper returns the original status to the public manager.

Special NACK handling

NACK is considered recoverable if bus teardown succeeds.

The public manager performs:

```
Receive I2C_ERR_NACK
↓
Complete STOP/teardown
↓
If completion succeeds:
    State = READY
    Return I2C_ERR_NACK
↓
If completion fails:
    State = ERROR
    Return cleanup failure
```

This prevents a normal missing-target or rejected-command condition from permanently locking the driver.

Timeouts, overrun, arbitration loss and uncertain bus failures place the driver in:

```
I2C_STATE_ERROR
```

until recovery is explicitly requested.

15. Recovery Architecture

The public recovery function is:

```
i2c_recover()
```

It is not simply a software-state clear operation.

Recovery performs a deterministic I2C1 peripheral reset and reconfiguration.

Recovery-state policy

```
RESET or invalid saved configuration  
→ I2C_ERR_NOT_INIT
```

```
BUSY_TX, BUSY_RX or BUSY_TX_RX  
→ I2C_ERR_BUSY
```

```
READY  
→ I2C_OK, no action required
```

```
ERROR  
→ reset and reconfigure I2C1
```

The peripheral-only recovery flow is:

```
Keep state as ERROR  
↓  
Reset I2C1 through RCC  
↓  
Reapply OAR1, timing, ACK and POS configuration  
↓  
Enable I2C1  
↓
```

```
Check SR2.BUSY
    ↓
If bus idle:
    State = READY
    Return I2C_OK

If bus still busy:
    State = ERROR
    Return I2C_ERR_BUS
```

Recovery does not reconfigure GPIO because the pins and clocks remain configured from successful initialization.

Recovery limitation

Resetting I2C1 only recovers the STM32 peripheral.

It does not physically release a target that is holding SDA or SCL low.

A future physical bus-clear function may:

- Disable I2C1.
- Temporarily configure SCL and SDA as open-drain GPIO.
- Pulse SCL up to nine times.
- Generate a manual STOP condition.
- Restore AF4.
- Reinitialize the peripheral.

This physical recovery mechanism is outside the V1 scope.

16. Main Design Decisions

Decision 1: Keep public managers separate from register primitives

Reason:

- State and policy remain visible at the public layer.
- Register operations remain reusable.
- Future interrupt or DMA implementations can preserve the same API and transaction model.

Decision 2: Use separate receive functions for 1, 2 and N bytes

Reason:

The STM32F4 I²C peripheral requires materially different ACK, POS and STOP sequences. Combining them into one heavily conditional function would make protocol review more difficult and increase the risk of teardown mistakes.

Decision 3: Keep STOP generation atomic

`i2c_generate_stop()` only sets the STOP bit.

Reason:

Setting STOP cannot itself produce a software timeout. Waiting for completion is a different responsibility and remains in `i2c_complete_transaction()`.

Decision 4: Make the transmit helper stop before STOP

`i2c_transmit_phase()` ends after final BTF.

Reason:

- `i2c_write()` can terminate with STOP.
 - `i2c_write_read()` can continue with repeated START.
 - No Boolean mode argument is required.
 - Transaction policy stays with the public manager.
-

Decision 5: Reuse the existing receive helpers after repeated START

Reason:

The receive sequence after the read address is the same whether the START was initial or repeated. The hardware determines repeated-START behaviour because the controller still owns the bus.

Decision 6: Treat NACK differently from hardware-context failures

Reason:

NACK may simply mean:

- The target is absent.
- The target is not ready.
- The address is wrong.
- A command or data byte was rejected.

If STOP cleanup succeeds, the controller remains usable and should return to `READY`.

Decision 7: Lock the driver after uncertain hardware failures

Reason:

After a timeout, overrun or failed completion, changing only the software state to `READY` would not prove that ACK, POS, START, STOP, ADDR, DR or the internal I²C state machine are clean.

The driver therefore enters `ERROR` and requires deterministic peripheral recovery.

Decision 8: Share a private peripheral-configuration function

Both initialization and recovery call:

```
i2c_reset_and_configure_peripheral()
```

Reason:

- Register configuration is defined in one place.
 - Initialization and recovery retain separate public contracts.
 - Recovery does not unnecessarily reconfigure clocks and GPIO.
 - Calling `i2c_init()` from `i2c_recover()` is avoided.
-

Decision 9: Check hardware BUSY before transaction admission

Reason:

Software `READY` does not guarantee that the physical bus is idle.

The check avoids issuing an initial START into a bus that is already active or stuck.

The check is not placed in `i2c_generate_start()` because that primitive must also support repeated START.

Decision 10: Use a conservative polling V1

Reason:

The priority of V1 is understandable, testable and deterministic protocol operation.

Interrupt-driven state transitions, DMA, timer-based deadlines, retry policies and physical bus clearing are deferred until the polling implementation is hardware validated.

17. Example Application Use

```
#include "i2c_driver.h"

int main(void)
{
    i2c_status_t status;

    const i2c_config_t i2c_config =
    {
        .clock_speed = I2C_CLOCK_STANDARD_SAFE_HZ
    };

    system_clock_init();

    status = i2c_init(&i2c_config);

    if (status != I2C_OK)
    {
        while (1)
        {
            /* Initialization fault */
        }
    }

    while (1)
    {
        /*
         * Application transactions:
         *
         * i2c_write()
         * i2c_read()
         * i2c_write_read()
         */
    }
}
```



```
}  
}
```

A typical register read is:

```
uint8_t register_address = 0x10U;  
uint8_t data[2];  
  
status = i2c_write_read(  
    target_address,  
    &register_address,  
    1U,  
    data,  
    2U);
```

This produces:

```
START  
target address + write  
register address  
repeated START  
target address + read  
two data bytes  
STOP
```

18. V1 Acceptance Tests

The driver is code-complete only after the final compile checks pass.

Hardware acceptance should cover:

Write tests

- One-byte write
- Multi-byte write
- Address NACK
- NACK on a transmitted data byte

Read tests

- One-byte read

- Two-byte read
- Three-byte read
- Read longer than three bytes

Combined write-read tests

- One-byte write followed by one-byte read
- One-byte write followed by two-byte read
- One-byte write followed by multi-byte read
- Multi-byte command followed by read

State and error tests

- Transaction requested before initialization
- Transaction requested while another transaction is active
- Transaction requested while state is ERROR
- Address NACK followed by a successful transaction
- Forced timeout followed by successful `i2c_recover()`
- Hardware BUSY detected at transaction admission
- BUSY remaining after recovery
- Repeated transactions without stale ACK or POS state

Signal-level verification

Use an oscilloscope or logic analyzer to confirm:

- Actual SCL frequency
- START and STOP generation
- Address direction bit
- ACK/NACK behaviour
- No STOP between write and read phases
- Correct repeated START
- Correct number and ordering of transferred bytes

19. Future V2 Direction

The next implementation stage may retain the public API while replacing the polling internals.

Possible V2 improvements include:

- Interrupt-driven event and error handling
- DMA data transfer
- Hardware-timer or system-tick deadlines
- Transaction descriptors

- Non-blocking completion callbacks
- Retry policy for arbitration loss
- Physical GPIO bus clearing
- Dynamic APB1 clock detection
- Fast-mode support
- Multiple I²C peripheral instances
- Thread-safe or RTOS-aware transaction serialization

The central architecture remains useful:

```
Public transaction manager
      ↓
Protocol state machine
      ↓
Register-level hardware operations
```

The polling V1 therefore acts both as a functional driver and as the reference behaviour for future asynchronous implementations.